Embeddings & Search

Bag-of-words, TF-IDF, BM25, semantic embeddings, hybrid search fusion, and evaluation metrics

Concept Guide 7 of 9 | Read before: Embeddings & Search Zero-to-Hero lab | Companion to the Zero-to-Hero series

---

1. Bag-of-Words: Text as a Vector

Before any math can be done on text, it must become numbers. The simplest approach: count how many times each vocabulary word appears in a document.

DIAGRAM — Bag-of-words vectors

```
vocab:  [app, crash, password, refund, ...]
'app keeps crashing'  -> [1, 1, 0, 0, ...]   (word counts, order IGNORED)
'refund my refund'    -> [0, 0, 0, 2, ...]
```

COMMON MISUNDERSTANDINGS

- Bag-of-words completely ignores word ORDER: 'dog bites man' and 'man bites dog' produce identical vectors. This is why more advanced methods (TF-IDF weighting, then embeddings) are layered on top.

PRACTICE THIS → **Embeddings_Search_Zero_to_Hero.ipynb Ch.2**

2. TF-IDF: Weighting Words by Rarity

Raw word counts over-weight common words like 'the' and 'my.' TF-IDF fixes this by multiplying a word's frequency in one document by how RARE it is across all documents.

FORMULA — TF-IDF

$$\text{TF-IDF(word, doc)} = \text{count(word, doc)} * \text{IDF(word)}$$

$$\text{IDF(word)} = \log\left((1 + N) / (1 + \text{doc\_freq(word)}) \right) + 1$$

N = total documents; doc_freq = how many documents contain the word at least once

WORKED EXAMPLE — Worked example, N=10 documents

'my' appears in 8 of 10 docs: IDF = log(11/9) + 1 = log(1.222) + 1 = 0.201 + 1 = 1.201 (low, common)

'refund' appears in 2 of 10 docs: IDF = log(11/3) + 1 = log(3.667) + 1 = 1.299 + 1 = 2.299 (high, rare)

A document mentioning 'refund' twice gets score 2 * 2.299 = 4.598 for that word,

while 'my' appearing twice only gets 2 * 1.201 = 2.402 — rarer words dominate the score.

PRACTICE THIS → **Embeddings_Search_Zero_to_Hero.ipynb Ch.4**

3. BM25: The Industry-Standard Keyword Ranker

BM25 improves on TF-IDF with two refinements: term-frequency SATURATION (the 10th occurrence of a word adds much less than the 2nd) and length NORMALIZATION (long documents don't unfairly win just by being long).

FORMULA — The BM25 score for one term

```
score = IDF(term) * [ f*(k1+1) ] / [ f + k1*(1 - b + b*dl/avgdl) ]
```

f = term frequency in this doc; dl = doc length; avgdl = average doc length; k1~1.5, b~0.75 are tuned constants

MENTAL MODEL

As f (term frequency) grows very large, the score approaches a ceiling of IDF*(k1+1) instead of growing forever — this is the saturation effect. The (dl/avgdl) term shrinks the score for documents that are longer than average, correcting for the fact that longer documents naturally contain more word repeats.

COMMON MISUNDERSTANDINGS

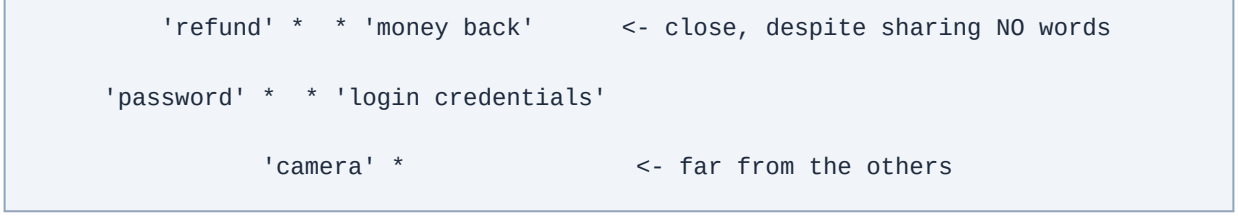
- BM25's defaults (k1=1.5, b=0.75) are battle-tested across decades of information retrieval research — start there and only tune with a real evaluation set showing improvement.

PRACTICE THIS → **Embeddings_Search_Zero_to_Hero.ipynb Ch.5**

4. Semantic Embeddings and Cosine Similarity

An embedding maps text to a dense vector where MEANING-similar text ends up geometrically close — even with zero shared words. Comparing embeddings uses cosine similarity (see the NumPy & Linear Algebra guide for the formula).

DIAGRAM — Semantic space (illustrative)



MENTAL MODEL

Keyword search matches SPELLINGS. Embeddings match IDEAS. A query for 'get my money back' can retrieve a document about 'refund policy' purely because their learned vectors point in a similar direction — how that vector space is LEARNED is covered in the Embedding Model Training guide.

PRACTICE THIS → **Embeddings_Search_Zero_to_Hero.ipynb Ch.6**

5. Hybrid Search: Combining Both Signals

Keyword search and semantic search have complementary strengths. Hybrid search fuses both scores after normalizing each to a common [0,1] scale.

FORMULA — Min-max normalization and fusion

```
normalized_score = (score - min_score) / (max_score - min_score)
combined = alpha * semantic_score + (1 - alpha) * keyword_score
```

alpha is a tuning knob: push toward 1 for natural-language queries, toward 0 for exact-code/ID search

MENTAL MODEL

Keyword search is unbeatable for exact terms (product codes, error strings, names). Semantic search catches paraphrases and synonyms. Fusing both usually beats either alone, which is why production search systems rarely use just one.

PRACTICE THIS → **Embeddings_Search_Zero_to_Hero.ipynb Ch.7**

6. Evaluating Search: Precision@k and MRR

'Looks good' isn't a metric. Against a small labeled set (query -> the correct document), two standard measures quantify search quality.

FORMULA — Precision@k and Reciprocal Rank

```
precision@k = 1 if the correct doc is anywhere in the top k
                    results, else 0
reciprocal_rank = 1 / (rank of the first correct result)
MRR = mean(reciprocal_rank) across all test queries
```

MRR rewards ranking the correct answer HIGHER, not just somewhere in the top-k

WORKED EXAMPLE — Worked example

Correct result found at rank 1: reciprocal_rank = 1/1 = 1.0

Correct result found at rank 3: reciprocal_rank = 1/3 = 0.333

Correct result not found in top-k at all: reciprocal_rank = 0

PRACTICE THIS → **Embeddings_Search_Zero_to_Hero.ipynb Ch.9**